XYZ
Data Engineering Case

Data quality, privacy controls and unbounded ingestion on Google Cloud

In-scope raw assets
XYZ-benjaminpiscoya.XYZ.modules
XYZ-benjaminpiscoya.XYZ.module_Services

This document explains the challenge, the target design, the rationale for keeping DQ in BigQuery, and the specific session_id privacy treatment.

Audience: CTO, Data Architect, Senior Data Engineer
Repository target: https://github.com/benwalkers/data_engineering_case_01.git

Executive summary

This XYZ case is as a governed streaming data product rather than as a simple SQL cleansing exercise. The raw event tables are large, continuously growing and carry both business-quality defects and privacy concerns.

The recommended target architecture uses Pub/Sub and Dataflow for unbounded ingestion, applies Sensitive Data Protection specifically to session_id before curated exposure, lands trusted and rejected records in BigQuery, and uses Dataplex to scan and govern the landed assets.

A BigQuery DQ zone is intentionally part of the design because failed records are still operational evidence. They must remain queryable for trend analysis, root-cause analysis, producer feedback and replay selection. Cloud Storage remains useful only as the optional cold archive of the untouched reject payload.

1. Problem definition

XYZ captures behavioural interaction events from retailer experiences where product content modules are shown to end users. These events power downstream content analytics, client reporting, and marketing insight.

The raw layer is large and continuously growing. In previous exploration, the two principal raw tables in scope were XYZ-benjaminpiscoya.XYZ.modules and XYZ-benjaminpiscoya.XYZ.module_Services. The target solution therefore needs to support unbounded ingestion rather than a one-off batch clean-up.

The case also contains a specific privacy requirement: session_id is a field that should not be exposed raw in curated analytical outputs.

2. Raw assets and schema access

The exact schema should be surfaced from BigQuery INFORMATION_SCHEMA rather than copied manually into a design document. That keeps the report accurate even if the source schema evolves.

The architecture below therefore uses a selected schema view: only fields that materially affect the design are highlighted.

Recommended schema access SQL

```
SELECT table_name, row_count
FROM `XYZ-benjaminpiscoya.XYZ.__TABLES__`;

SELECT table_name, column_name, data_type
FROM `XYZ-benjaminpiscoya.XYZ.INFORMATION_SCHEMA.COLUMNS`
WHERE table_name IN ('modules', 'module_Services')
ORDER BY table_name, ordinal_position;
```

Selected fields that drive the solution

Raw table	Selected fields	Why they matter
Modules	date, session_id, distributor_id, product_id, module_id, module_pos, views, view_time, clicks, blocking_flags	Partitioning, clustering, grain, privacy treatment and core business-rule validation.
module_Services	date, session_id, distributor_id, product_id, lang, currency, gbp_price, sis_id, blocking_flags	Partitioning, privacy treatment and the distinction between blocking and non-blocking data quality checks.

3. Main design requirements

- Support continuous ingestion and avoid a design that only works for static one-time data loads.
- Separate hard business failures from softer analytical warnings.
- Protect session_id before curated analytical exposure.
- Keep failed records queryable instead of throwing them away or hiding them only in a bucket.
- Version the rule set and schema expectations in Git, with a path to automated promotion and post-load governance.

4. Target architecture

The target-state architecture is intentionally split across four concerns: ingestion, inline enforcement, storage, and governance/observability.

Pub/Sub and Dataflow are used because the long-term design must support unbounded event ingestion. Sensitive Data Protection is used specifically in the streaming path to tokenise session_id before the record reaches the curated analytical layer.

BigQuery contains three analytical operating surfaces: raw, curated and DQ. Dataplex scans the landed BigQuery assets and adds governance, catalogue and quality monitoring on top of the runtime pipeline.

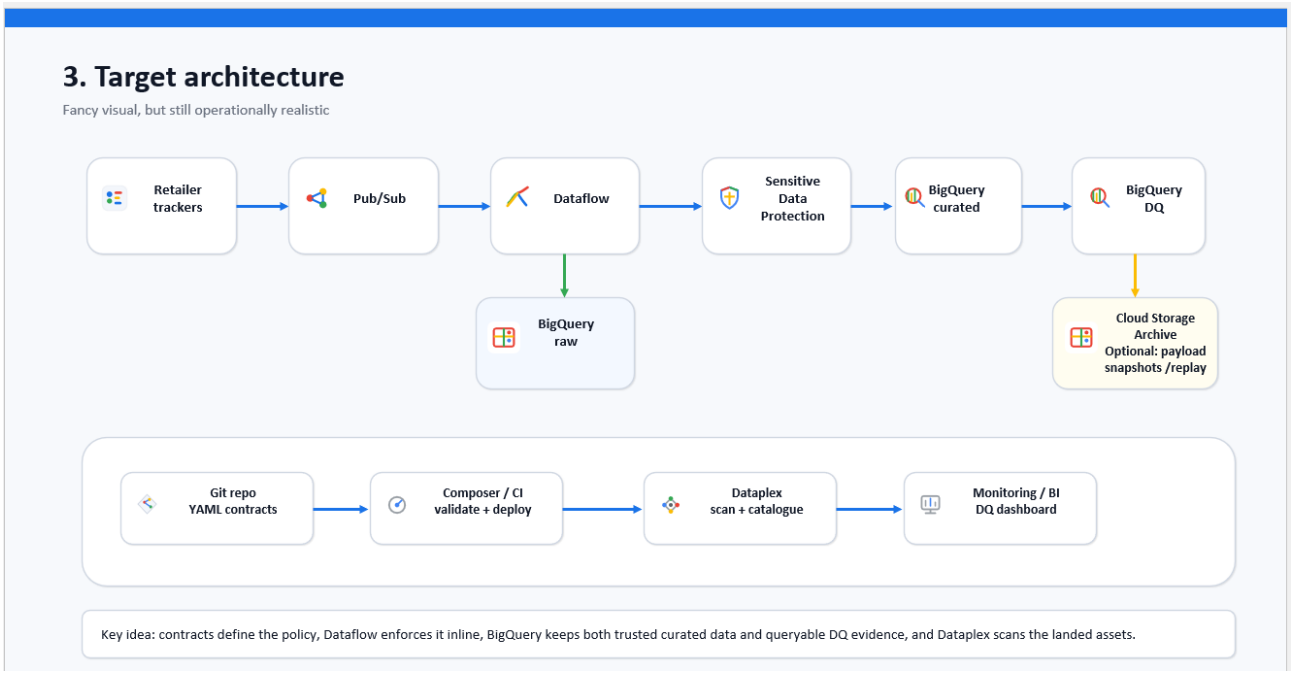

Figure 1. Proposed Target architecture view using Google Cloud component-style cards.

5. PII handling for this exact case

This design uses one specific PII-risk field: session_id. No abstract examples are needed. The architecture simply treats session_id as a field that may be necessary for behavioural journey analysis but must not be exposed raw in curated reporting outputs.

The practical control is deterministic pseudonymisation. session_id is ingested into the restricted raw zone, then the Dataflow pipeline calls Sensitive Data Protection so that the record written to curated tables carries session_token instead of the original session_id.

Deterministic tokenisation matters because the same pseudonymous person journey must remain joinable across records and across time windows.

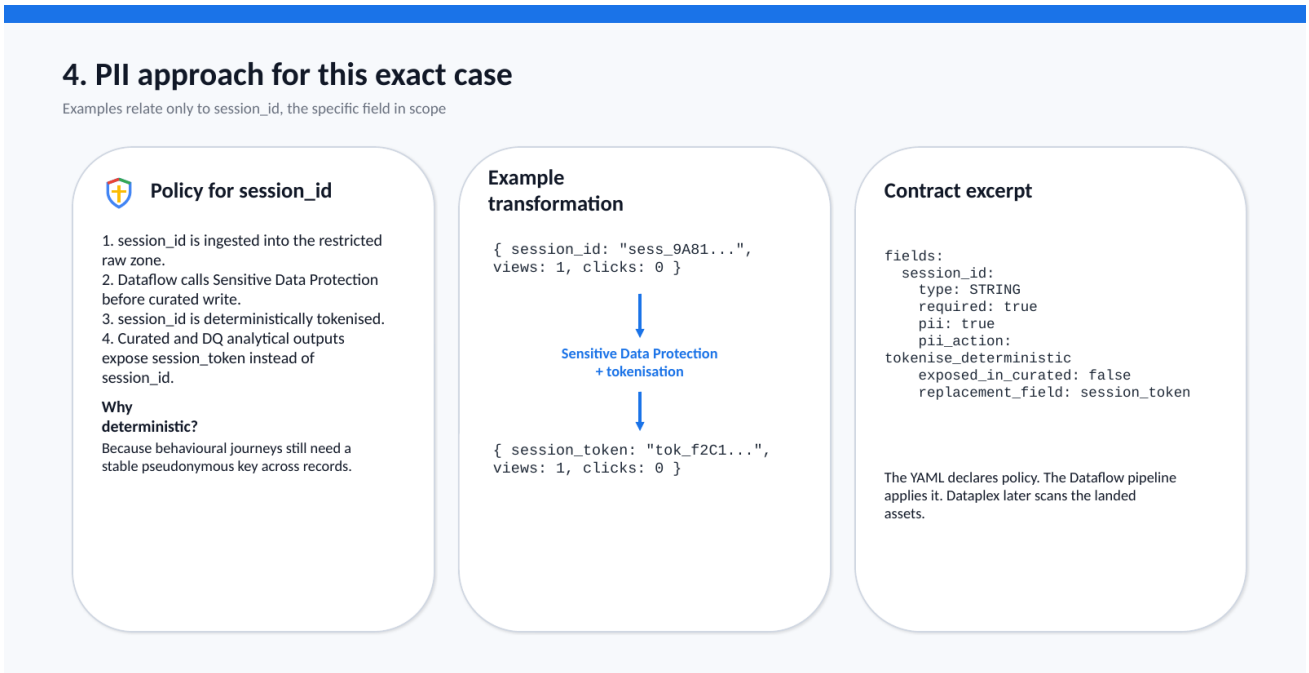

Figure 2. session_id treatment and example contract excerpt.

Example YAML contract snippet

```
fields:
  session_id:
    type: STRING
    required: true
    pii: true
    pii_action: tokenise_deterministic
    exposed_in_curated: false
    replacement_field: session_token
```

6. Why DQ belongs in BigQuery

A common question is why failed records are kept in BigQuery instead of being pushed only to a bucket. The answer is that failed records are not useless data (business rules); they are operational evidence.

The platform needs to trend failure rates, analyse which rules are failing, join failures to producer dimensions such as date, distributor_id or product_id, and target replay runs when a rule changes. BigQuery is the right GCP service for that operating surface because it supports direct SQL analysis, views, dashboards and rapid investigation.

Cloud Storage still has a role, but it is a secondary one. It is best used only to preserve the untouched raw reject payload for lower-cost cold retention or forensic replay.

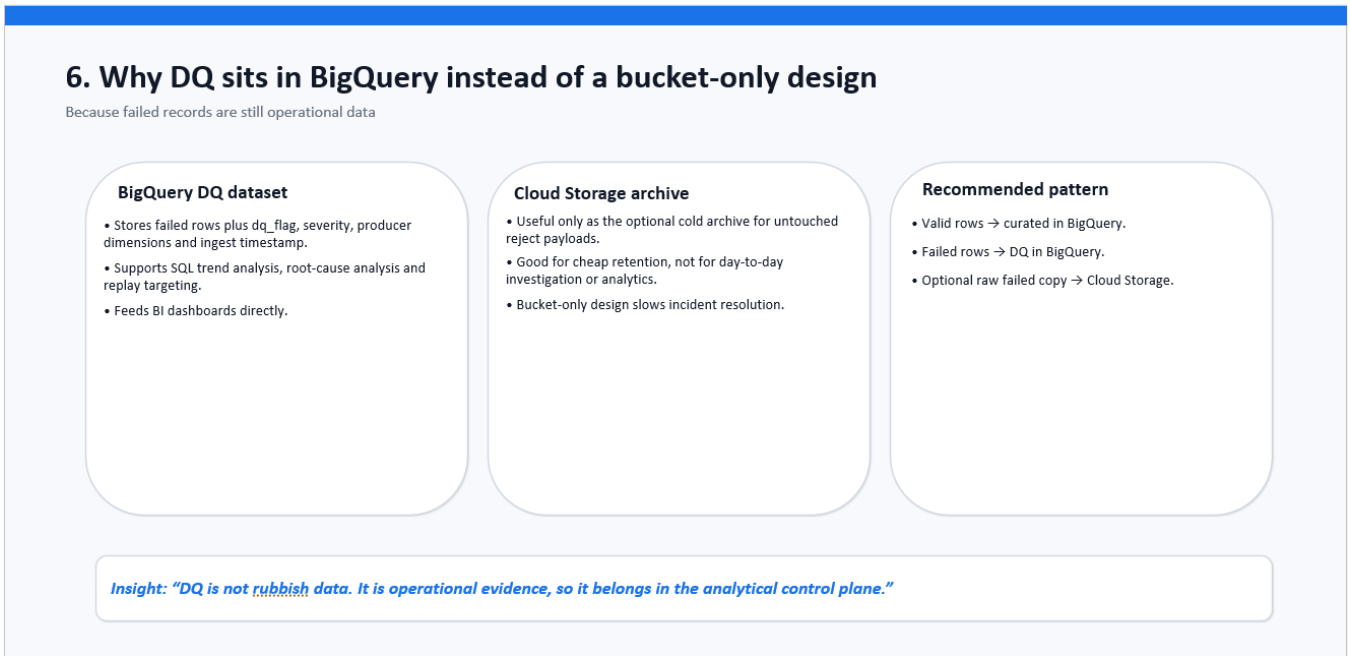

Figure 3. BigQuery DQ versus bucket-only comparison.

Recommended DQ pattern

- Valid rows → curated tables in BigQuery
- Failed rows → DQ tables in BigQuery
- Optional untouched failed payload → Cloud Storage archive

7. Why the design uses YAML contracts, DLP and Dataplex together

The architecture is deliberately layered rather than tool-exclusive.

YAML contracts in Git are the source of truth for schema, required fields, business rule severity and the session_id privacy declaration. This makes the rule set reviewable, testable and versioned.

Sensitive Data Protection is the dedicated service for inspection and de-identification. It solves the privacy task that a plain contract file cannot solve by itself.

Dataplex then complements the solution by scanning and governing the BigQuery assets after they land. In other words, contracts define the expectation, Dataflow enforces it inline, and Dataplex observes and governs the landed data product.

Operational flow

Git contracts	Composer / CI	Dataflow runtime	Dataplex scan	BI / monitoring

8. Proposed repository structure

```
data_engineering_case/
├── docs/
│   ├── XYZ_strong_architecture_report.docx
│   └── architecture_diagrams/
├── contracts/
│   ├── modules.contract.yaml
│   └── module_services.contract.yaml
├── validation/
│   ├── validate_contracts.py
│   └── rule_catalogue.yaml
├── dataflow/
│   ├── streaming_pipeline.py
│   └── pii_session_tokenisation.py
├── bigquery/
│   ├── ddl_raw.sql
│   ├── ddl_curated.sql
│   ├── ddl_dq.sql
│   └── monitoring_views.sql
├── composer/dags/
│   └── XYZ_contract_deploy.py
└── dataplex/
    └── dq_scan_config/
```